

Columbia University
Department of Electrical Engineering
EE E4321. Problem Set #8. Memory design. PLAs.
Due: November 25, 2025, 5 PM EST by electronic submission

Please carefully follow these instructions. Please submit your solutions to Problem 1 below as a single PDF file attachment through Courseworks.

For Design Project Part III, please submit your writeup as a single PDF file attachment submitted by only one person of your two-person team. Please note clearly in the document the name of the two team members in your group.

To submit the layout, please stream out your design according to the instructions on the class website and attached to your submission as well.

1. (This problem should be completed individually.) In this problem, I want you to consider the implementation of a controller using PLAs. We will take advantage of the tool **espresso** to optimize our logic.

Consider a controller defined by the following VHDL description:

```
architecture rtl of controller is
    subtype state_type is std_ulogic_vector(0 to 3);
    constant s0: state_type := "0001";
    constant s1: state_type := "0010";
    constant s2: state_type := "0100";
    constant s3: state_type := "1000";
    signal state, next_state: state_type;
    signal con1, con2, con3: std_ulogic;
    signal out1, out2: std_ulogic;
    signal clk;

begin
    state_logic: process(state, con1, con2, con3) is
    begin
        out1 <= '0';
        out2 <= '0';
```

```

case state is
  when s0 =>
    out1 <= '0';
    out2 <= '0';
    next_state <= s1;
  when s1 =>
    out1 <= '1';
    if con1 = '1' then
      next_state <= s2;
    else
      next_state <= s1;
    end if;
  when s2 =>
    out2 <= '1';
    next_state <= s3;
  when s3 =>
    if con2 = '0' then
      next_state <= s3;
    elsif con3 = '0' then
      out1 <= '0';
      next_state <= s2;
    else
      next_state <= s1;
    end if;
  end case;
end process state_logic;

state_register: process (clk) is
begin
  if (clk = '1' and not(clk'stable)) then
    state <= next_state;
  end if;
end process state_register;
end architecture rtl;

```

Implement this finite-state machine as a PLA and a (four-bit) flip-flop. Present your results as (hand-drawn) schematics. Don't worry about sizing

Signal	Direction	Description
phi1	input	phase 1 clock
phi2	input	phase 2 clock
mem_read	input	enable the memory for read
mem_write	input	enable the memory for write
iobus<0:7>	bidirectional	data bus
addr<0:2>	input	address

or simulating your design. To design the PLA, use **espresso** for two-level logic optimization. You can assess the espresso man pages on the input file format by typing:

```
man -s 5 espresso
```

Additional documentation on **espresso** can also be found on the main espresso man pages:

```
man espresso
```

The next problem is part of your final design project; you should work on this part of the problem set with your design-project partner.

Design Project Part III. You need to design the memory for your final design project. Your memory stores 8 8-bit (one-byte) words and has the following interface to the rest of your core:

Please precharge your memory with $\overline{phi2}$ and qualify both the wordline and the write with the *phi1* clock. Use the **mem_read** signal to tristate the read driver. Only one level of decoding should be necessary (no predecoder). Please use the TSMC-provided SRAM layout found in the library **arrayLib** in `/courses/ee4321/arrayLib`.

In the **arrayLib** library, there is a cell **065_d499_M4_v1d1_demo_array_4x4**. This is a 8x4 SRAM array (despite the name of the cell); 8 rows, 4 columns. You will find a schematic there as well as the layout. What you need to do is modify this to make an 8x8 array. This will involve instantiating two

more instances of `065_d499_M4_v1d1_x4` and moving the `row_edge` cells and corner cells to the right. You will also need to add three more instances of the `wells_strap` cells.

To run LVS, you need to do one special setup on your Calibre environment. From Calibre → Setup → Netlist Export, you need to put in the "Include File" field, the following file:

```
"/courses/ee4321/tech/tsmc65/sram/065_d499_M4_v1d1_x1.cdl".
```

This is the netlist file for the SRAM cell itself. With this setup, you will be able run LVS and DRC using the special rules and models for the SRAM designs.

In order to do the SRAM simulation in Spectre, you will need to include another model library.

- In ADE-L, go to Setup → Model Libraries and add the following file path at the bottom in a new line entry:

```
"/courses/ee4321/tech/tsmc65/sram/CLN65GPLUS_sr_v1d0.scs".
```

This is necessary as the SRAM transistors use different models than those used for general digital logic. Spectre will fail to run otherwise.

- Under the "Section" column next to the file path, select `"tt_sr"`. This is the typical-typical corner specifically for the SRAM.
- Click apply and run the simulation.

To complete your memory design, I expect:

- Sized schematics
- Layout of your memory design (remember to begin by making a tiling diagram and then stick layouts of your cells)
- Spectre results for the read and write delays of your SRAM. Verify that reads do not disturb the contents of the cells and that you will always be able to write your cell.
- Use Spectre FX to verify functionality for a larger number of patterns.